

分治之美

排序与二分 · 7题 · C++ & Python 双语对照

NQ101 AcWing 785 AcWing 785

AcWing 785

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

void quicksort(long long nums[], int left, int right)
{
    if (left >= right)
        return;
    //选取分割数
    long long target = nums[left]; //选取第一个快排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //分治
    quicksort(nums, left, j); //j左边的都是不比Target大的
    quicksort(nums, j + 1, right); // j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostream的输入输出缓存
    cin.tie(0); //cin.tie(0)来解除cin与cout的绑定,0表示NULL

    int n;
    //创建动态数组
    cin >> n;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    quicksort(numbers, 0, n - 1); //注意边界

    for (int i = 0; i < n; i++)
        cout << numbers[i] << " ";

    delete numbers;
    return 0;
}

```

Python

AcWing 785

NQ102 AcWing 786 AcWing 786

AcWing 786

- 输入

见原题
- 输出

见原题
- 范围

见原题

输入	输出
见原题	见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

long long quickFind(long long nums[], int left,
int right, int k)
{
    if (left == right)
        return nums[left]; //只有一个元素，左开右
    闭
    //选取分割数
    long long target = nums[left]; //选取第一个快
    排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //判断i与k的关系
    int sumofLeft = j - left + 1; //计算左边区间
    数字个数，用于与k比较

    if (k <= sumofLeft) //分治
        quickFind(nums, left, j, k); //j左边的都
    是不比Target大的
    else
        quickFind(nums, j + 1, right, k - sumof
    Left); // j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostrea
    m的输入输出缓存
    cin.tie(0); //cin.tie(0)来
    解除cin与cout的绑定,0表示NULL

    int n, k;
    //创建动态数组
    cin >> n >> k;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    //查找第k小数，输出之
    cout << quickFind(numbers, 0, n - 1, k) <<
    endl; //注意边界

    delete numbers;
    return 0;
}

```

Python

AcWing 786

NQ103 AcWing 787 AcWing 787

AcWing 787

输入 见原题**输出** 见原题**范围** 见原题输入
见原题输出
见原题**思路** 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
const int N = 100007;
int n;
int numbers[N], tmp[N];

void mergeSort(int nums[], int left, int right)
{
    if (left >= right) return;

    int mid = left + (right - left) / 2; // 防止溢出

    mergeSort(nums, left, mid), mergeSort(nums, mid + 1, right); // 先调用归并排序
    // 合二为一
    int k = 0, i = left, j = mid + 1; // 左右两部分数组的起点

    while (i <= mid && j <= right) // 扫描还未抵达终点
        if (nums[i] <= nums[j]) // 把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else tmp[k++] = nums[j++];

    while (i <= mid) tmp[k++] = nums[i++]; // 把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; // 把右边剩余的部分插入tmp

    // 把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) nums[i] = tmp[k];
}

int main()
{
    // 数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    mergeSort(numbers, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", numbers[i]);

    return 0;
}

```

Python

AcWing 787

NQ104 AcWing 788 AcWing 788

AcWing 788

输入

见原题

输出

见原题

范围

见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
typedef long long LL;

const int N = 100007;
int n;
int numbers[N], tmp[N];
//最大的数可能会是  $n^2/2$ 
LL mergeSort(int nums[], int left, int right)
{
    if (left >= right) return 0; //如果左指针越过右指针, 则返回

    int mid = left + (right - left) / 2; //防止溢出

    LL result = mergeSort(nums, left, mid) + mergeSort(nums, mid + 1, right); //先调用归并排序
    //合二为一
    int k = 0, i = left, j = mid + 1; //左右两部分数组的起点

    while (i <= mid && j <= right) //扫描还未抵达终点
        if (nums[i] <= nums[j]) //把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else {
            //计算逆序对的个数mid-i+1
            result += mid - i + 1;
            tmp[k++] = nums[j++];
        }

    while (i <= mid) tmp[k++] = nums[i++]; //把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; //把右边剩余的部分插入tmp

    //把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) numbers[i] = tmp[k];

    return result;
}

int main()
{
    //数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    cout << mergeSort(numbers, 0, n - 1) << endl;

    return 0;
}

```

Python

AcWing 788

NQ105 AcWing 789

AcWing 789

AcWing 789

- 输入

见原题
- 输出

见原题
- 范围

见原题

输入	输出
见原题	见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```

#include <iostream>
#include <algorithm>
using namespace std;
const int N = 100007;

int nums[N];

int main()
{
    // 第一行包含整数n和q，表示数组长度和询问个数。
    int n, q;
    scanf("%d%d", &n, &q);
    // 第二行包含n个整数（均在1~10000范围内），表示完
    // 整数组。
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    // 接下来q行，每行包含一个整数k，表示一个询问元
    // 素。
    while(q--){
        //读入要查找的数，并且二分查找其范围
        int k;
        scanf("%d",&k);// 读入k

        int left = 0, right = n-1;//初始化left和ri
        ght
        while (left < right) //模板2
        {
            int mid = left + right>>1;
            if (nums[mid]>=k) right = mid;//模板
            else left = mid +1;
        }
        //left就是左端点值
        //判断是否找得到
        if (nums[left]!=k) cout<<"-1 -1"<<endl;//
        找不到x
        else
        {
            cout<<left<<" ";
            //继续寻找右边界
            left=0, right=n-1;//重新初始化left,rii
            ght,进行第二次二分搜索
            while (left < right)
            {
                int mid = left+right+1>>1;
                if (nums[mid]<=k) left=mid;
                else right = mid -1;
            }
            cout<<left<<endl;
        }
    }

    return 0;

// int bsearch_1(int left, int right)
// {
//     while (left < right)
//     {
//         int mid = left + right >> 1;
//         if (check(mid)) right = mid;
//         else left = mid + 1;

```

Python

AcWing 789

```
//     }
//     return left;
// }

// int bsearch_2(int left, int right)
// {
//     while (left < right)
//     {
//         int mid = left + right + 1 >> 1;
//         if (check(mid)) left = mid;
//         else right = mid - 1;
//     }
//     return left;
// }
```

NQ106 AcWing 790 AcWing 790

AcWing 790

输入 见原题

输出 见原题

范围 见原题

输入
见原题

输出
见原题

思路 见AcWing原题描述

技巧 参考模板库 CONTEXT.md

C++

```
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    double number;
    scanf("%lf", &number);
    double left = -10000, right = 10000;
    while (right - left >= 1e-8)
    {
        double mid = (right + left) / 2; //二分
        if (mid * mid * mid >= number)
            right = mid;
        else
            left = mid;
    }
    //输出left
    printf("%lf", left);
}
```

Python

AcWing 790

NQ107 AcWing 727 AcWing 727

AcWing 727

输入 见原题**输出** 见原题**范围** 见原题

输入

见原题

输出

见原题

思路 见AcWing原题描述**技巧** 参考模板库 CONTEXT.md

C++

```
#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int cx = n / 2, cy = n / 2;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            if (abs(i - cx) + abs(j - cy) <= n / 2)
                cout << '*';
            else cout << ' ';
        cout << endl;
    }

    return 0;
}
```

Python

AcWing 727